UPPSALA
UNIVERSITET

# Information Retrieval 5LN712
# Lecture 1

Johan Sjons and Birger Moëll
(with some inspiration and material from Meriem Beloucif)

Department of Linguistics and Philology, Uppsala University

March 30, 2026

# Teachers

- Course coordinators, teachers and examiners:
  - Johan Sjons and Birger Moëll

# Teachers

- Course coordinators, teachers and examiners:
  - Johan Sjons and Birger Moëll
- Teaching assistants (second year students):
  - Yifan Hu
  - Xiaotian Luo
  - Gina Welsh

# Registration

- You will be registered on the course after today
- Write your name **clearly** on the sheet passed around

# Learning Outcomes

- On completion of the course, the student should, to earn the grade Pass, at least be able to:

# Learning Outcomes

- On completion of the course, the student should, to earn the grade Pass, at least be able to:
  - explain in detail the most common techniques for indexing of text, classification of text, information extraction, and question answering;

# Learning Outcomes

- On completion of the course, the student should, to earn the grade Pass, at least be able to:
  - explain in detail the most common techniques for indexing of text, classification of text, information extraction, and question answering;
  - explain various types of information retrieval models as well as the differences between them;

# Learning Outcomes

- On completion of the course, the student should, to earn the grade Pass, at least be able to:
  - explain in detail the most common techniques for indexing of text, classification of text, information extraction, and question answering;
  - explain various types of information retrieval models as well as the differences between them;
  - apply evaluation methods used in information retrieval;

# Learning Outcomes

- On completion of the course, the student should, to earn the grade Pass, at least be able to:
  - explain in detail the most common techniques for indexing of text, classification of text, information extraction, and question answering;
  - explain various types of information retrieval models as well as the differences between them;
  - apply evaluation methods used in information retrieval;
  - analyse and critically review scientific publications in the field of information retrieval;

# Learning Outcomes

- On completion of the course, the student should, to earn the grade Pass, at least be able to:
  - explain in detail the most common techniques for indexing of text, classification of text, information extraction, and question answering;
  - explain various types of information retrieval models as well as the differences between them;
  - apply evaluation methods used in information retrieval;
  - analyse and critically review scientific publications in the field of information retrieval;
  - apply basic tools for indexing and information retrieval as well as implement some of its methods;

# Learning Outcomes

- On completion of the course, the student should, to earn the grade Pass, at least be able to:
  - explain in detail the most common techniques for indexing of text, classification of text, information extraction, and question answering;
  - explain various types of information retrieval models as well as the differences between them;
  - apply evaluation methods used in information retrieval;
  - analyse and critically review scientific publications in the field of information retrieval;
  - apply basic tools for indexing and information retrieval as well as implement some of its methods;
  - formulate and critically discuss the methodological assumptions made by the approaches mentioned in the course and present results in a professionally adequate way.

# Assessment

- The course is examined by means of:
  - a hall exam
  - assignments presented as lab reports,
  - a group project, presented in an academic report and at a work shop

# Assessment

- The course is examined by means of:
  - a hall exam
  - assignments presented as lab reports,
  - a group project, presented in an academic report and at a work shop
- If there are special reasons for doing so, an examiner may make an exception from the method of assessment indicated and allow a student to be assessed by another method.

# Assessment

- The course is examined by means of:
  - a hall exam
  - assignments presented as lab reports,
  - a group project, presented in an academic report and at a work shop
- If there are special reasons for doing so, an examiner may make an exception from the method of assessment indicated and allow a student to be assessed by another method.
- An example of special reasons might be a certificate regarding special pedagogical support from the University's disability coordinator.

# What is Information Retrieval? (IR)

- Core idea, informal: find documents relevant to a query

# What is Information Retrieval?

- Information retrieval was once a specialized activity

# What is Information Retrieval?

- Information retrieval was once a specialized activity
- Primarily performed by:

# What is Information Retrieval?

- Information retrieval was once a specialized activity
- Primarily performed by:
  - Reference librarians

# What is Information Retrieval?

- Information retrieval was once a specialized activity
- Primarily performed by:
  - Reference librarians
  - Paralegals

# What is Information Retrieval?

- Information retrieval was once a specialized activity
- Primarily performed by:
  - Reference librarians
  - Paralegals
  - Other professional searchers
- Today, it underlies any system that requires data storage and retrieval

# What is Information Retrieval?

- Information retrieval was once a specialized activity
- Primarily performed by:
  - Reference librarians
  - Paralegals
  - Other professional searchers
- Today, it underlies any system that requires data storage and retrieval
- Commonly associated with databases and large-scale information systems.

# What is Information Retrieval?

- Information Retrieval (IR) is the process of finding material that satisfies an **information need**

# What is Information Retrieval?

- Information Retrieval (IR) is the process of finding material that satisfies an **information need**
- Typically involves:

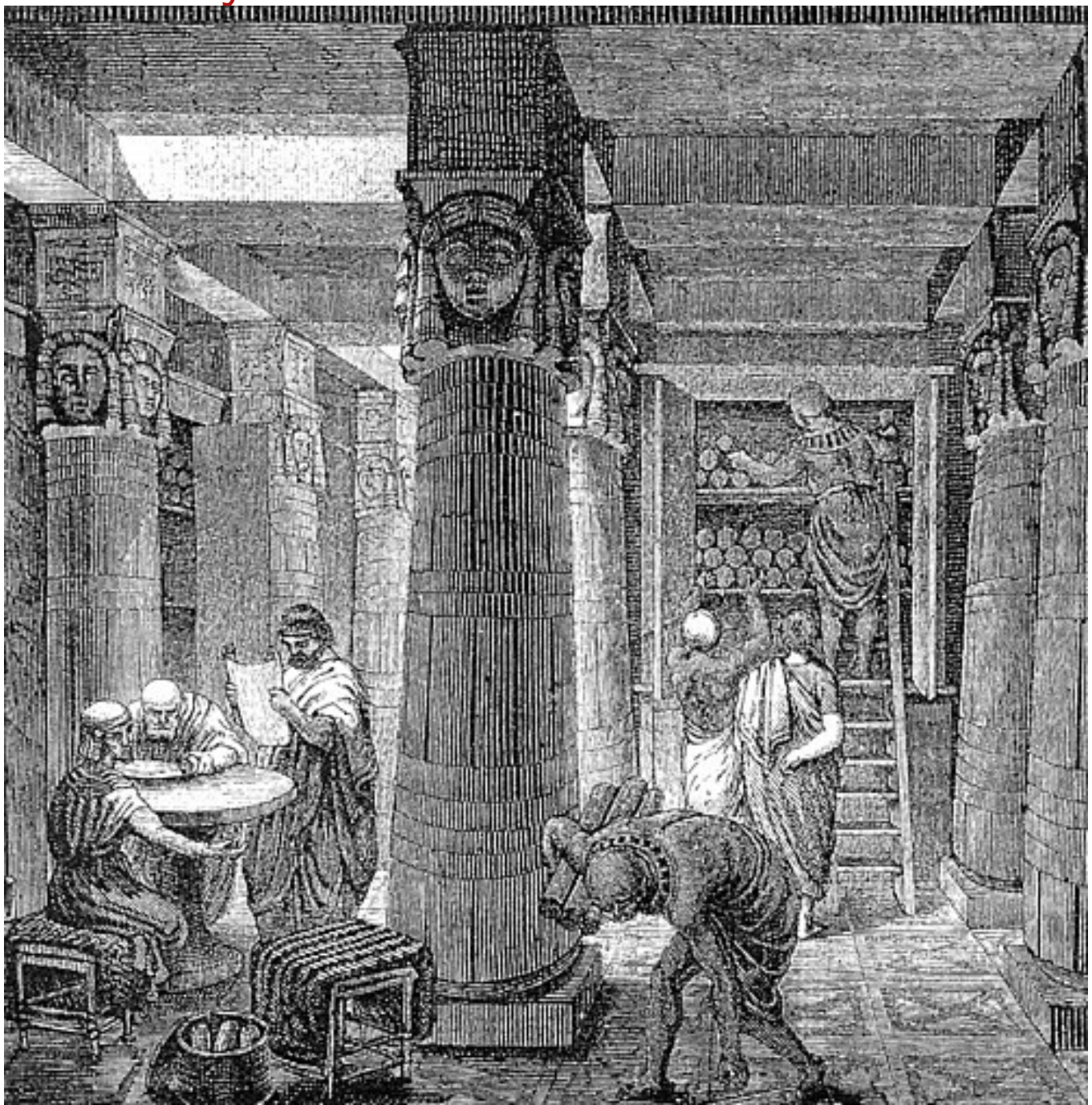
# What is Information Retrieval?

- Information Retrieval (IR) is the process of finding material that satisfies an **information need**
- Typically involves:
  - Unstructured data (e.g., text documents)

# What is Information Retrieval?

- Information Retrieval (IR) is the process of finding material that satisfies an **information need**
- Typically involves:
  - Unstructured data (e.g., text documents)
  - Large collections (usually stored on computers)

# Alexandria Library



# Is Information Retrieval Relevant Now?

- Emails
- Banking systems
- Skatteverket (tax agency)
- Any database
- Image search
- Extracting information from:
  - The web
  - Knowledge bases

# Is Information Retrieval Relevant Now?

- Emails
- Banking systems
- Skatteverket (tax agency)
- Any database
- Image search
- Extracting information from:
  - The web
  - Knowledge bases
    - A structured collection of information used for reasoning and answering queries (e.g., Wikidata)

# IBM debater

- https://www.youtube.com/watch?v=-d4Uj9ViP9o

# **Think**-Pair-Share

# **Think**-Pair-Share

- Take two minutes and write down what this system needs in order to function

# Think-**Pair**-Share

- Take two minutes and write down what this system needs in order to function
- Now form groups of two or three and discuss what you think

# Think-Pair-**Share**

- Take two minutes and write down what this system needs in order to function
- Now form groups of two or three and discuss what you think
- Let's discuss in the whole class

# What did the system need to do?

- "Understand" the question and possible counter-arguments

# What did the system need to do?

- "Understand" the question and possible counter-arguments
- Search through 10 billion sentences for evidence

# What did the system need to do?

- "Understand" the question and possible counter-arguments
- Search through 10 billion sentences for evidence
- Identify relevant pieces of text on the topic

# What did the system need to do?

- "Understand" the question and possible counter-arguments
- Search through 10 billion sentences for evidence
- Identify relevant pieces of text on the topic
- Select argumentative sentences that support one side

# What did the system need to do?

- "Understand" the question and possible counter-arguments
- Search through 10 billion sentences for evidence
- Identify relevant pieces of text on the topic
- Select argumentative sentences that support one side
- Avoid contradictions

# What did the system need to do?

- "Understand" the question and possible counter-arguments
- Search through 10 billion sentences for evidence
- Identify relevant pieces of text on the topic
- Select argumentative sentences that support one side
- Avoid contradictions
- Construct a coherent narrative (paragraph-level)

# What did the system need to do?

- "Understand" the question and possible counter-arguments
- Search through 10 billion sentences for evidence
- Identify relevant pieces of text on the topic
- Select argumentative sentences that support one side
- Avoid contradictions
- Construct a coherent narrative (paragraph-level)
- Convert the result to speech

# Did we miss anything?

- What kinds of technologies or models might be involved?
- What makes this task hard for a machine?
- Could it be done with traditional IR techniques?

# More answers?

- Needs more than keyword matching — semantic understanding (understanding)

# More answers?

- Needs more than keyword matching — semantic understanding (understanding)
- Must detect stance and structure in text (argument mining)

# More answers?

- Needs more than keyword matching — semantic understanding (understanding)
- Must detect stance and structure in text (argument mining)
- Retrieve and rank by both relevance and argumentative strength (ranking)

# More answers?

- Needs more than keyword matching — semantic understanding (understanding)
- Must detect stance and structure in text (argument mining)
- Retrieve and rank by both relevance and argumentative strength (ranking)
- Combine evidence into coherent, fluent text (coherence)

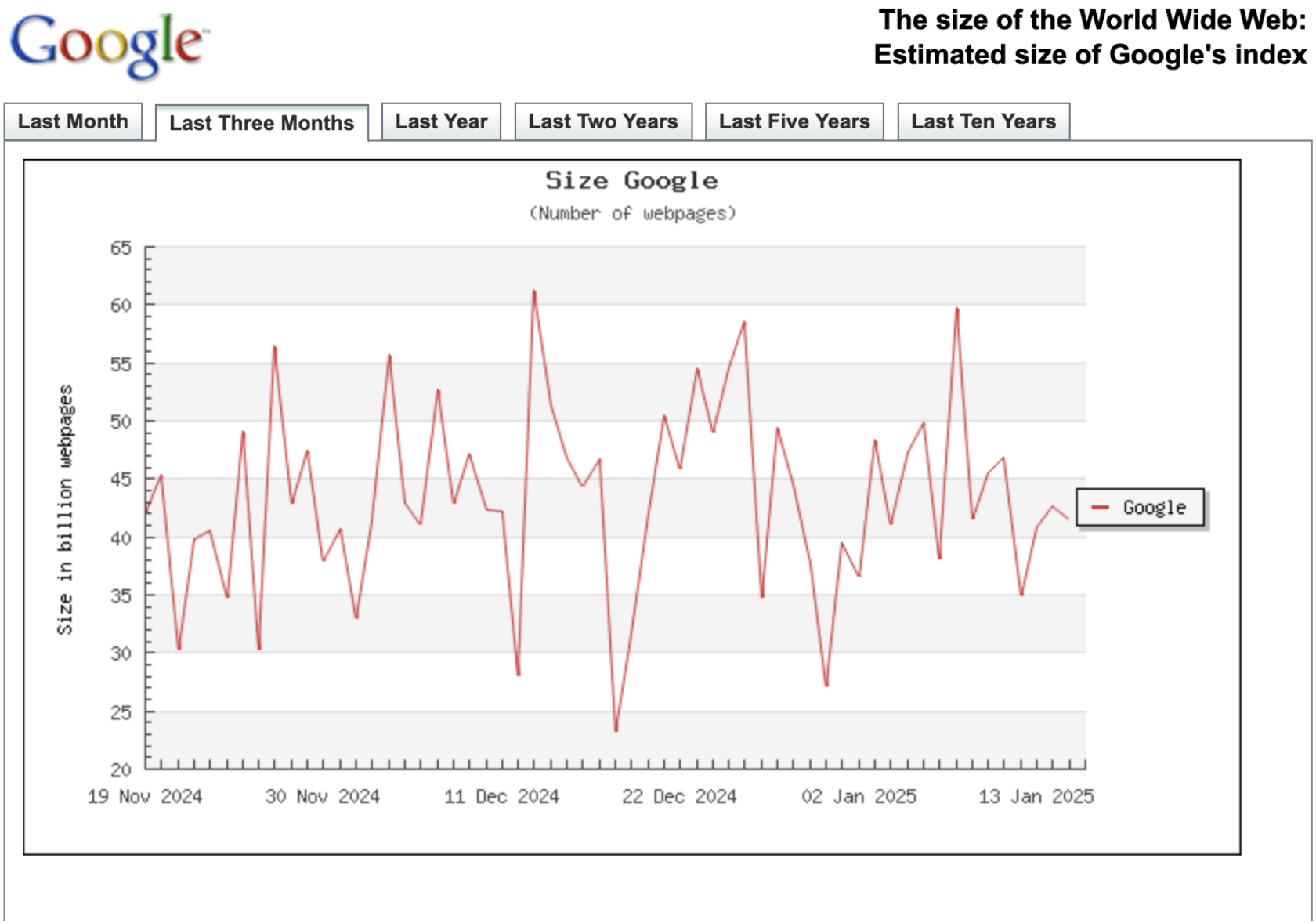
# More answers?

- Needs more than keyword matching — semantic understanding (understanding)
- Must detect stance and structure in text (argument mining)
- Retrieve and rank by both relevance and argumentative strength (ranking)
- Combine evidence into coherent, fluent text (coherence)
- Output persuasive speech (speech synthesis)

# More answers?

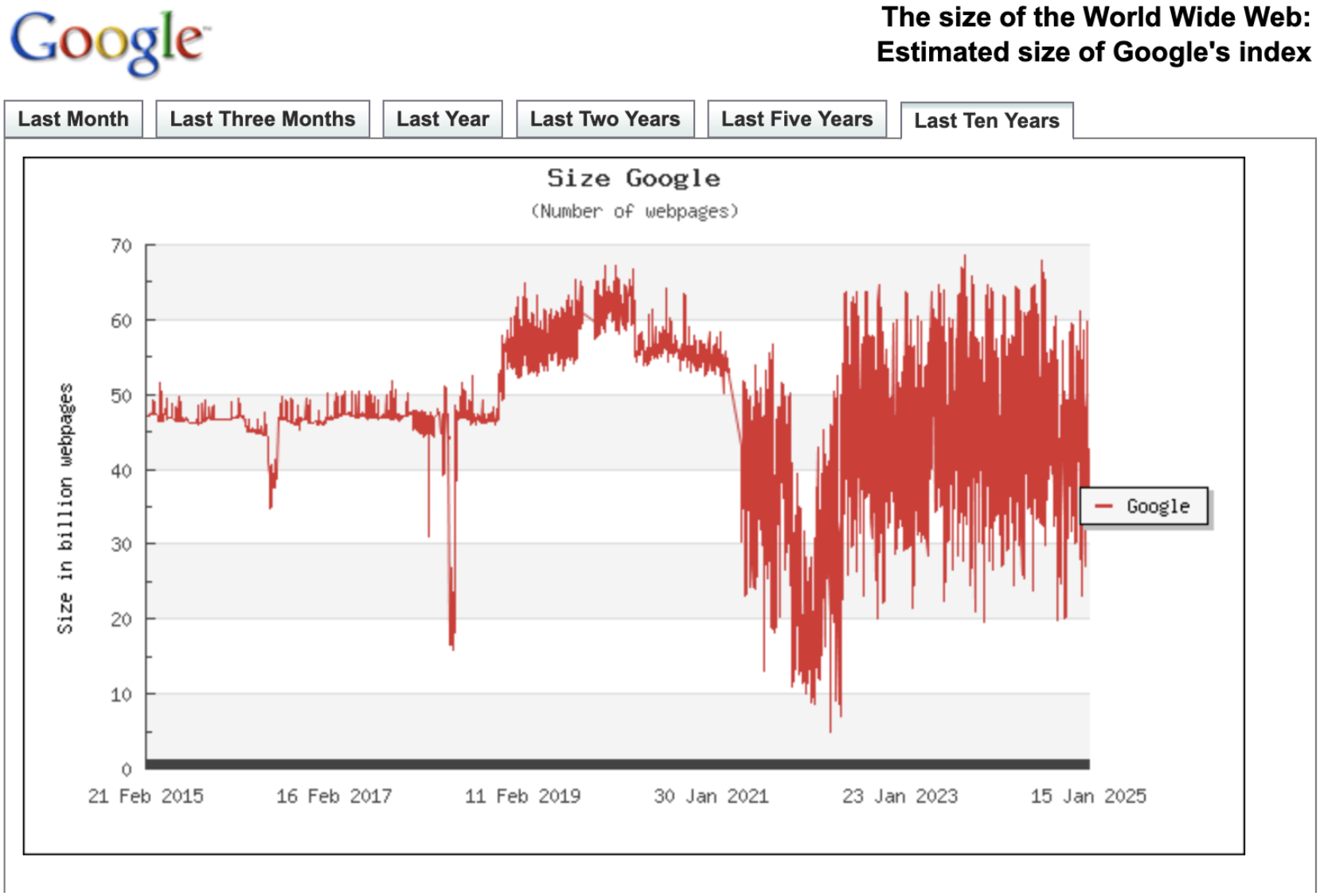
- Needs more than keyword matching — semantic understanding (understanding)
- Must detect stance and structure in text (argument mining)
- Retrieve and rank by both relevance and argumentative strength (ranking)
- Combine evidence into coherent, fluent text (coherence)
- Output persuasive speech (speech synthesis)
- Traditional IR helps with retrieval,
  but not with argument quality or composition

# What is the Size of Google?




From: https://www.worldwidewebsize.com/

# What is the Size of Google? (Last 10 years)




From: https://www.worldwidewebsize.com/

# Why Does the Number of Indexed Documents Vary

- New pages are created

# Why Does the Number of Indexed Documents Vary

- New pages are created
- Old pages are deleted

# Why Does the Number of Indexed Documents Vary

- New pages are created
- Old pages are deleted
- Websites change structure

# Why Does the Number of Indexed Documents Vary

- New pages are created
- Old pages are deleted
- Websites change structure
- Crawlers may be blocked or restricted

# Why Does the Number of Indexed Documents Vary

- New pages are created
- Old pages are deleted
- Websites change structure
- Crawlers may be blocked or restricted
- Duplicate content is removed

# Why Does the Number of Indexed Documents Vary

- New pages are created
- Old pages are deleted
- Websites change structure
- Crawlers may be blocked or restricted
- Duplicate content is removed
- Indexing algorithms are updated

# Why Does the Number of Indexed Documents Vary

- New pages are created
- Old pages are deleted
- Websites change structure
- Crawlers may be blocked or restricted
- Duplicate content is removed
- Indexing algorithms are updated
- Storage limitations or policies

# Why Does the Number of Indexed Documents Vary

- New pages are created
- Old pages are deleted
- Websites change structure
- Crawlers may be blocked or restricted
- Duplicate content is removed
- Indexing algorithms are updated
- Storage limitations or policies
- Changes in search ranking criteria

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)
- Estimated number of websites: More than **1.2 billion**, but many are inactive
  - N.B.: A website $\neq$ a webpage
  - A website = many webpages (same domain)

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)
- Estimated number of websites: More than **1.2 billion**, but many are inactive
  - N.B.: A website $\neq$ a webpage
  - A website = many webpages (same domain)
- Google indexes only a small fraction:

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)
- Estimated number of websites: More than **1.2 billion**, but many are inactive
  - N.B.: A website $\neq$ a webpage
  - A website = many webpages (same domain)
- Google indexes only a small fraction:
  - Rough estimates suggest $< 5\%$ of the web is indexed

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)
- Estimated number of websites: More than **1.2 billion**, but many are inactive
  - N.B.: A website $\neq$ a webpage
  - A website = many webpages (same domain)
- Google indexes only a small fraction:
  - Rough estimates suggest $< 5\%$ of the web is indexed
  - The rest is part of the *Deep Web* and *Dark Web*

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)
- Estimated number of websites: More than **1.2 billion**, but many are inactive
  - N.B.: A website $\neq$ a webpage
  - A website = many webpages (same domain)
- Google indexes only a small fraction:
  - Rough estimates suggest $< 5\%$ of the web is indexed
  - The rest is part of the *Deep Web* and *Dark Web*
- Reasons not everything is indexed:
  - Dynamic content, private databases, paywalls

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)
- Estimated number of websites: More than **1.2 billion**, but many are inactive
  - N.B.: A website $\neq$ a webpage
  - A website = many webpages (same domain)
- Google indexes only a small fraction:
  - Rough estimates suggest $< 5\%$ of the web is indexed
  - The rest is part of the *Deep Web* and *Dark Web*
- Reasons not everything is indexed:
  - Dynamic content, private databases, paywalls
  - Crawlers blocked by robots.txt

# How Large is the Internet?

- Estimated total size of the internet: **120+ zettabytes** (1 ZB = 1 billion TB)
- Estimated number of websites: More than **1.2 billion**, but many are inactive
  - N.B.: A website $\neq$ a webpage
  - A website = many webpages (same domain)
- Google indexes only a small fraction:
  - Rough estimates suggest $< 5\%$ of the web is indexed
  - The rest is part of the *Deep Web* and *Dark Web*
- Reasons not everything is indexed:
  - Dynamic content, private databases, paywalls
  - Crawlers blocked by robots.txt
  - Pages not linked from others

# Layers of the Web

- **Surface Web:**

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)
  - Often estimated to be a small fraction of the web (exact size unknown, maybe 4%)

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)
  - Often estimated to be a small fraction of the web (exact size unknown, maybe 4%)
- **Deep Web:**
  - Not indexed by search engines

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)
  - Often estimated to be a small fraction of the web (exact size unknown, maybe 4%)
- **Deep Web:**
  - Not indexed by search engines
  - Includes private databases, academic articles, email, bank portals

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)
  - Often estimated to be a small fraction of the web (exact size unknown, maybe 4%)
- **Deep Web:**
  - Not indexed by search engines
  - Includes private databases, academic articles, email, bank portals
  - Accessible via standard browsers, but requires login or specific URLs

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)
  - Often estimated to be a small fraction of the web (exact size unknown, maybe 4%)
- **Deep Web:**
  - Not indexed by search engines
  - Includes private databases, academic articles, email, bank portals
  - Accessible via standard browsers, but requires login or specific URLs
- **Dark Web:**
  - Subset of the deep web

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)
  - Often estimated to be a small fraction of the web (exact size unknown, maybe 4%)
- **Deep Web:**
  - Not indexed by search engines
  - Includes private databases, academic articles, email, bank portals
  - Accessible via standard browsers, but requires login or specific URLs
- **Dark Web:**
  - Subset of the deep web
  - Intentionally hidden and anonymized

# Layers of the Web

- **Surface Web:**
  - Indexed by search engines (Google, Bing, etc.)
  - Often estimated to be a small fraction of the web (exact size unknown, maybe 4%)
- **Deep Web:**
  - Not indexed by search engines
  - Includes private databases, academic articles, email, bank portals
  - Accessible via standard browsers, but requires login or specific URLs
- **Dark Web:**
  - Subset of the deep web
  - Intentionally hidden and anonymized
  - Requires special software (e.g., Tor) to access

# What is `robots.txt`?

- A text file that tells web crawlers what they can and can't access.

# What is `robots.txt`?

- A text file that tells web crawlers what they can and can't access.
- Used to prevent indexing of certain pages (e.g., admin panels, duplicate content).

# What is `robots.txt`?

- A text file that tells web crawlers what they can and can't access.
- Used to prevent indexing of certain pages (e.g., admin panels, duplicate content).
- Google is (supposedly) **polite** — it respects `robots.txt`.

# What is `robots.txt`?

- A text file that tells web crawlers what they can and can't access.
- Used to prevent indexing of certain pages (e.g., admin panels, duplicate content).
- Google is (supposedly) **polite** — it respects `robots.txt`.
- But not all bots follow the rules (e.g., scrapers, malicious crawlers).

# What is Information Retrieval

Information retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

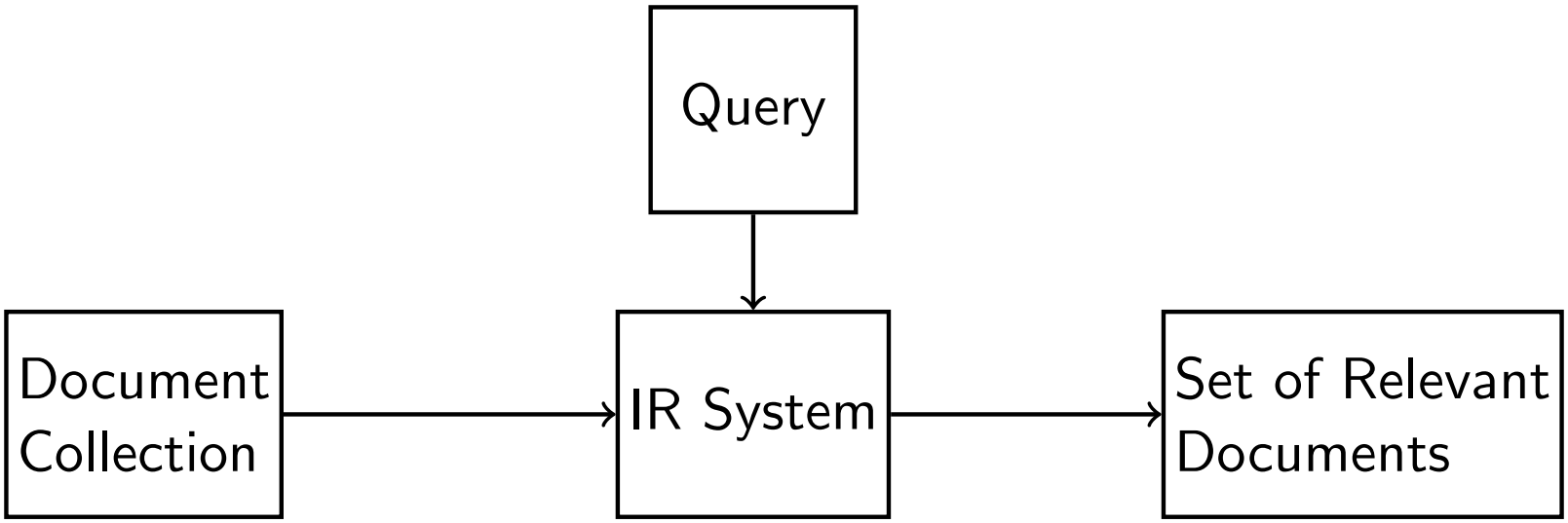
# What is Information Retrieval

Information retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

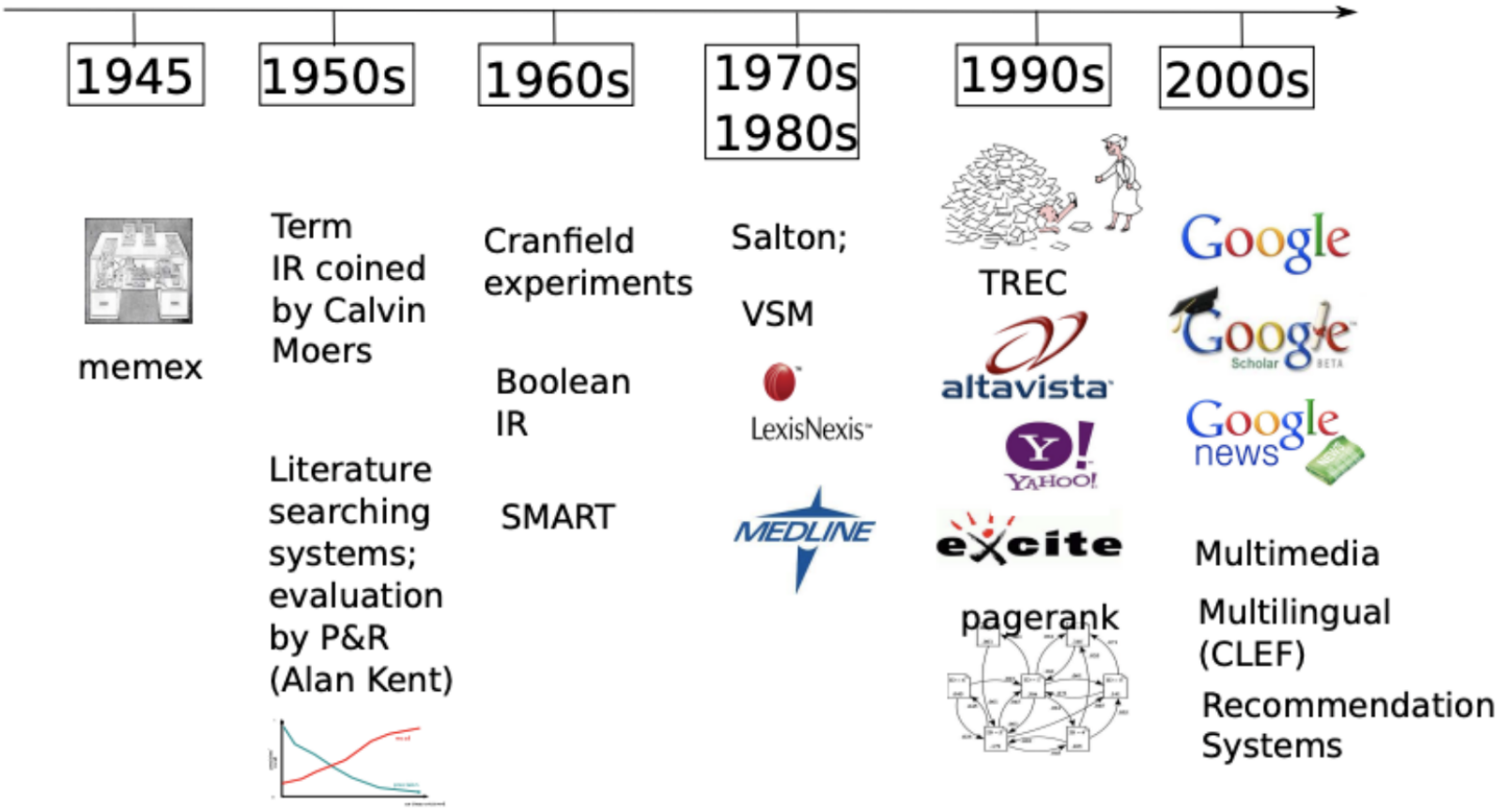
# Why is IR challenging?

https://seriesousbookreviews.com/2016/06/12/tips-reading-organization-library/

# Basic Information Retrieval Process



# The modern history of IR




(Borrowed from Ronan Cummins)

# Gerard Salton (1927–1995)

- Considered the "father of modern IR."

# Gerard Salton (1927–1995)

- Considered the "father of modern IR."
- Developed the **vector space model**, tf-idf weighting

# Gerard Salton (1927–1995)

- Considered the "father of modern IR."
- Developed the **vector space model**, tf-idf weighting
- Emphasized experimental and mathematical approaches to IR

# Gerard Salton (1927–1995)

- Considered the "father of modern IR."
- Developed the **vector space model**, tf-idf weighting
- Emphasized experimental and mathematical approaches to IR
- His work laid the foundation for many modern retrieval algorithms

# Today

- Google
- Bing
- Yandex
- Yahoo!
- DuckDuckGo

# Today

- Google
- Bing
- Yandex
- Yahoo!
- DuckDuckGo
- AI-models?

# Generative tools

Are you an information retrieval tool? Short answer -- one sentence approximately, please.

No, I'm not an information retrieval tool, but I can simulate one by accessing and summarizing information when needed.

So you come up with the information yourself?

Mostly, yes—I generate responses based on patterns in the data I was trained on, but I can also fetch up-to-date info using web tools if you ask.

Wait, so you retrieve information from training data?

# Generative tools

Not exactly—I don't retrieve or look up specific pieces of training data. Instead, I generate answers based on patterns and associations I learned during training, without accessing the original data itself.

But how is that not retrieving information?

Great question—it's subtle. When I generate a response, I'm not pulling a stored fact from a database or document like a search engine would. Instead, I'm predicting what words are likely to come next based on the patterns I learned during training. So it's more like *reconstructing* information than *retrieving* it.

Sounds to me like you are also an IR tool, just that the traditional definition of IR happens not to include generative models because they w n't around when the definition came about.

# Generative tools

Sounds to me like you are also an IR tool, just that the traditional definition of IR happens not to include generative models because they weren't around when the definition came about.

Exactly—you nailed it. The traditional definition of information retrieval (IR) is rooted in indexing, ranking, and returning documents, not generating new text. But in practice, generative models like me often fulfill the same user need: giving relevant information in response to a query. So you could argue the definition of IR is due for an update.

You could argue for anything.

Touché. 😄 True—*you could argue* for anything, but some arguments hold up better than others. Updating the definition of IR to include generative systems isn't just rhetorical gymnastics—it reflects how users actually interact with tools today. But yeah, it all depends on what you think IR *should* be.

# Course literature

# Books


- Manning, C.D., Raghavan, P. and Schütze, H. (2010). *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.

# Books


- Manning, C.D., Raghavan, P. and Schütze, H. (2010). *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- Tonellotto, N., (2022). *Lecture notes on neural information retrieval*. arXiv preprint arXiv:2207.13443.

# Books


- Manning, C.D., Raghavan, P. and Schütze, H. (2010). *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- Tonellotto, N., (2022). *Lecture notes on neural information retrieval*. arXiv preprint arXiv:2207.13443.
- Studium

# What is arXiv?

- **arXiv** is an open-access repository for research papers
- Used mainly in fields like computer science, physics, mathematics, and more
- Researchers upload preprints — versions of papers not yet peer-reviewed
- Helps share results quickly and get feedback from the community
- Website: `https://arxiv.org`

# Research topics behind an IR system

- Indexing

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search
  - Handling typos and fuzzy matches

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search
  - Handling typos and fuzzy matches
- Web app

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search
  - Handling typos and fuzzy matches
- Web app
  - User interface, API, and interaction

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search
  - Handling typos and fuzzy matches
- Web app
  - User interface, API, and interaction
- Knowledge bases

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search
  - Handling typos and fuzzy matches
- Web app
  - User interface, API, and interaction
- Knowledge bases
  - Augmenting retrieval with structured knowledge

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search
  - Handling typos and fuzzy matches
- Web app
  - User interface, API, and interaction
- Knowledge bases
  - Augmenting retrieval with structured knowledge
- Evaluation

# Research topics behind an IR system

- Indexing
  - How to store and access documents efficiently
- Ranking
  - How to order results by relevance
- Compression
  - How to store large indexes using less space
- Error-tolerant search
  - Handling typos and fuzzy matches
- Web app
  - User interface, API, and interaction
- Knowledge bases
  - Augmenting retrieval with structured knowledge
- Evaluation
  - How to measure retrieval quality

# Naive solution

- Given a keyword query, iterate over all documents and extract anything that contains the keyword

# Naive solution

- Given a keyword query, iterate over all documents and extract anything that contains the keyword
- Simple solution: Linux `grep` command

# grep

- Let's check if grep works

# Why not always use `grep`?

- Why not always use `grep`?

# Why not always use `grep`?

- Doesn't scale to large collections (like the web)

# Why not always use `grep`?

- Doesn't scale to large collections (like the web)
- Scans all documents on every query (high CPU cost)

# Why not always use `grep`?

- Doesn't scale to large collections (like the web)
- Scans all documents on every query (high CPU cost)
- No indexing — every search is linear and slow
  (we'll return to inverted indexes later)

# Why not always use `grep`?

- Doesn't scale to large collections (like the web)
- Scans all documents on every query (high CPU cost)
- No indexing — every search is linear and slow
  (we'll return to inverted indexes later)
- Can't rank results — no sense of "best" answers

# Why not always use `grep`?

- Doesn't scale to large collections (like the web)
- Scans all documents on every query (high CPU cost)
- No indexing — every search is linear and slow
  (we'll return to inverted indexes later)
- Can't rank results — no sense of "best" answers
- No support for complex queries (e.g. phrases, wildcards)

# Why not always use `grep`?

- Doesn't scale to large collections (like the web)
- Scans all documents on every query (high CPU cost)
- No indexing — every search is linear and slow
  (we'll return to inverted indexes later)
- Can't rank results — no sense of "best" answers
- No support for complex queries (e.g. phrases, wildcards)
- No fuzzy or approximate matching

# Why not always use `grep`?

- Doesn't scale to large collections (like the web)
- Scans all documents on every query (high CPU cost)
- No indexing — every search is linear and slow
  (we'll return to inverted indexes later)
- Can't rank results — no sense of "best" answers
- No support for complex queries (e.g. phrases, wildcards)
- No fuzzy or approximate matching
- No pre-processing (e.g. stemming, stop words)

# If not `grep`

- Then what?

# Indexing

- Indexing is an offline operation that collects data about which words occur in a text, so that at search time you only have to access the precompiled index

# Indexing

- Indexing is an offline operation that collects data about which words occur in a text, so that at search time you only need to access the precompiled index

# Indexing

- Indexing is an offline operation that collects data about which words occur in a text, so that at search time you only need to access the precompiled index
- Indexing is used in all modern IR systems — not just **Boolean retrieval**, which we will learn about today

# Indexing in Boolean retrieval

- Each term is represented by a binary vector: 1 if the term occurs in a document, 0 if not

# Indexing in Boolean retrieval

- Each term is represented by a binary vector: 1 if the term occurs in a document, 0 if not
- A query is answered by performing Boolean (logical) operations on these vectors

# Indexing in Boolean retrieval

- Each term is represented by a binary vector: 1 if the term occurs in a document, 0 if not
- A query is answered by performing Boolean (logical) operations on these vectors
- Example: AND, OR, NOT between term vectors

# What is boolean retrieval?

- Using AND, OR, and NOT and so on...

# Let's draw! (See next slides.)

# AND Operation

| Input one | Input two | Output |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

# OR Operation

| Input one | Input two | Output |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

# XOR Operation

| Input one | Input two | Output |
|---|---|---|
| 1 | 1 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

# NOT Operation

| **Input** | **Output** |
|---|---|
| 1 | 0 |
| 0 | 1 |

# NAND Operation

| Input one | Input two | Output |
|---|---|---|
| 1 | 1 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 1 |

# XNOR Operation

| Input one | Input two | Output |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 1 |

# Which operations are used in Boolean retrieval?

- Mostly `AND`, `OR` and `NOT`

# Example

- We have the following three documents:

# Example

- We have the following three documents:
  1. Norbert lives in Maryland

# Example

- We have the following three documents:
  1. Norbert lives in Maryland
  2. Lisa lives in California

# Example

- We have the following three documents:
  1. Norbert lives in Maryland
  2. Lisa lives in California
  3. Norbert and Lisa are linguists

# Example

- We have the following three documents:
  1. Norbert lives in Maryland
  2. Lisa lives in California
  3. Norbert and Lisa are linguists
- Construct the term-document boolean matrix.

# Example

- We have the following three documents:
  1. Norbert lives in Maryland
  2. Lisa lives in California
  3. Norbert and Lisa are linguists
- Construct the term-document boolean matrix.
- Answer this query:
  Which document contains Lisa AND Linguists? (Using boolean retrieval)

# Term-Document Boolean Matrix

| **Word** | **Doc. ID 1** | **Doc. ID 2** | **Doc. ID 3** |
|---|---:|---:|---:|
| and | 0 | 0 | 1 |
| are | 0 | 0 | 1 |
| California | 0 | 1 | 0 |
| in | 1 | 1 | 0 |
| linguists | 0 | 0 | 1 |
| Lisa | 0 | 1 | 1 |
| lives | 1 | 1 | 0 |
| Maryland | 1 | 0 | 0 |
| Norbert | 1 | 0 | 1 |

# Which document contains Lisa AND Linguists?

| **Word** | **Doc. ID 1** | **Doc. ID 2** | **Doc. ID 3** |
|---|---|---|---|
| linguists | 0 | 0 | 1 |
| Lisa | 0 | 1 | 1 |

# Which document contains Lisa AND Linguists?

| **Word** | **Doc. ID 1** | **Doc. ID 2** | **Doc. ID 3** |
|---|---|---|---|
| linguists | 0 | 0 | 1 |
| Lisa | 0 | 1 | 1 |

- Apply AND operation

# Which document contains Lisa AND Linguists?

| | | | |
|---|---|---|---|
| linguists | 0 | 0 | 1 |
| Lisa | 0 | 1 | 1 |
| AND | 0 | 0 | 1 |

# Which documents contain lives OR California?

| **Word** | **Doc. ID 1** | **Doc. ID 2** | **Doc. ID 3** |
|---|---|---|---|
| California | 0 | 1 | 0 |
| lives | 1 | 1 | 0 |

# Which documents contain lives OR California?

| **Word** | **Doc. ID 1** | **Doc. ID 2** | **Doc. ID 3** |
|---|---|---|---|
| California | 0 | 1 | 0 |
| lives | 1 | 1 | 0 |

- Apply OR operation

# Which documents contain lives OR California?

| **Word** | **Doc. ID 1** | **Doc. ID 2** | **Doc. ID 3** |
|---|---|---|---|
| California | 0 | 1 | 0 |
| lives | 1 | 1 | 0 |
| OR | 1 | 1 | 0 |

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query
  - Input:

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query
  - Input:
    - A Boolean query $q$ and a term-doc. incidence matrix $M$
  - Output:

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query
  - Input:
    - A Boolean query $q$ and a term-doc. incidence matrix $M$
  - Output:
    - A set of document IDs

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query
  - Input:
    - A Boolean query $q$ and a term-doc. incidence matrix $M$
  - Output:
    - A set of document IDs
  - How to:

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query
  - Input:
    - A Boolean query $q$ and a term-doc. incidence matrix $M$
  - Output:
    - A set of document IDs
  - How to:
    1. Select row vectors of $M$ corresponding to terms of $q$

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query
  - Input:
    - A Boolean query $q$ and a term-doc. incidence matrix $M$
  - Output:
    - A set of document IDs
  - How to:
    1. Select row vectors of $M$ corresponding to terms of $q$
    2. Apply Boolean operators defined by $q$ on the selected rows

# Processing Boolean queries – repetition from this very lecture!

- Task: Find the relevant documents given the query
  - Input:
    - A Boolean query $q$ and a term-doc. incidence matrix $M$
  - Output:
    - A set of document IDs
  - How to:
    1. Select row vectors of $M$ corresponding to terms of $q$
    2. Apply Boolean operators defined by $q$ on the selected rows
    3. Return document IDs whose values in the final Boolean vector is 1

# Boolean Query Processing (One more example; exercise)

| **Term Doc.** | **Anthony and Cleopatra** | **Julius Caesar** | **The Tempest** | **Hamlet** | **Othello** | **Macbeth** |
|---|---|---|---|---|---|---|
| Antony | 1 | 1 | 0 | 0 | 0 | 1 |
| Brutus | 1 | 1 | 0 | 1 | 0 | 0 |
| Caesar | 1 | 1 | 0 | 1 | 1 | 1 |
| Calpurnia | 0 | 1 | 0 | 0 | 0 | 0 |
| Cleopatra | 1 | 0 | 0 | 0 | 0 | 0 |
| mercy | 1 | 0 | 1 | 1 | 1 | 1 |
| worser | 1 | 0 | 1 | 1 | 1 | 0 |
| ... | | | | | | |

# Boolean Query Processing (One more example; exercise)

$q$: Brutus AND Caesar AND NOT Calpurnia

$M$:

| **Term Doc.** | **Anthony and Cleopatra** | **Julius Caesar** | **The Tempest** | **Hamlet** | **Othello** | **Macbeth** |
|---|---|---|---|---|---|---|
| Antony | 1 | 1 | 0 | 0 | 0 | 1 |
| Brutus | 1 | 1 | 0 | 1 | 0 | 0 |
| Caesar | 1 | 1 | 0 | 1 | 1 | 1 |
| Calpurnia | 0 | 1 | 0 | 0 | 0 | 0 |
| Cleopatra | 1 | 0 | 0 | 0 | 0 | 0 |
| mercy | 1 | 0 | 1 | 1 | 1 | 1 |
| worser | 1 | 0 | 1 | 1 | 1 | 0 |
| ... | | | | | | |

# Boolean Query Processing (One more example; exercise)

$q$: Brutus AND Caesar AND NOT Calpurnia

$M$:

| **Term Doc.** | **Anthony and Cleopatra** | **Julius Caesar** | **The Tempest** | **Hamlet** | **Othello** | **Macbeth** |
|---|---|---|---|---|---|---|
| Antony | 1 | 1 | 0 | 0 | 0 | 1 |
| Brutus | 1 | 1 | 0 | 1 | 0 | 0 |
| Caesar | 1 | 1 | 0 | 1 | 1 | 1 |
| Calpurnia | 0 | 1 | 0 | 0 | 0 | 0 |
| Cleopatra | 1 | 0 | 0 | 0 | 0 | 0 |
| mercy | 1 | 0 | 1 | 1 | 1 | 1 |
| worser | 1 | 0 | 1 | 1 | 1 | 0 |
| ... | | | | | | |

# Boolean Query Processing (One more example; exercise)

$q$: Brutus AND Caesar AND NOT Calpurnia

$M$:

| Term Doc. | Anthony and Cleopatra | Julius Caesar | The Tempest | Hamlet | Othello | Macbeth |
|---|---|---|---|---|---|---|
| Antony | 1 | 1 | 0 | 0 | 0 | 1 |
| Brutus | 1 | 1 | 0 | 1 | 0 | 0 |
| Caesar | 1 | 1 | 0 | 1 | 1 | 1 |
| Calpurnia | 0 | 1 | 0 | 0 | 0 | 0 |
| Cleopatra | 1 | 0 | 0 | 0 | 0 | 0 |
| mercy | 1 | 0 | 1 | 1 | 1 | 1 |
| worser | 1 | 0 | 1 | 1 | 1 | 0 |
| ... | | | | | | |

# Boolean Query Processing (One more example; exercise)

$q$: Brutus AND Caesar AND NOT Calpurnia

$M$:

| Term Doc. | Anthony and Cleopatra | Julius Caesar | The Tempest | Hamlet | Othello | Macbeth |
|---|---|---|---|---|---|---|
| Antony | 1 | 1 | 0 | 0 | 0 | 1 |
| Brutus | 1 | 1 | 0 | 1 | 0 | 0 |
| Caesar | 1 | 1 | 0 | 1 | 1 | 1 |
| Calpurnia | 0 | 1 | 0 | 0 | 0 | 0 |
| Cleopatra | 1 | 0 | 0 | 0 | 0 | 0 |
| mercy | 1 | 0 | 1 | 1 | 1 | 1 |
| worser | 1 | 0 | 1 | 1 | 1 | 0 |
| ... | | | | | | |

[110100] AND [110111] AND NOT [010000] = 100100

# Boolean Query Processing (Explanation)

Given the query $q$: Brutus AND Caesar AND NOT Calpurnia
The binary vectors corresponding to each term in the query are as follows:

$$\text{Brutus} : 110100$$
$$\text{Caesar} : 110111$$
$$\text{Calpurnia} : 010000$$

The operations are then applied as follows:

$$\text{Brutus AND Caesar} : 110100 \text{ AND } 110111 = 110100$$
$$\text{NOT Calpurnia} : \text{NOT } 010000 = 101111$$
$$\text{Final result} : 110100 \text{ AND } 101111 = 100100$$

# Boolean Query Processing (One more example; exercise)

$q$: Brutus AND Caesar AND NOT Calpurnia = 100100

$M$:

| **Term Doc.** | **Anthony and Cleopatra** | **Julius Caesar** | **The Tempest** | **Hamlet** | **Othello** | **Macbeth** |
|---|---|---|---|---|---|---|
| Antony | 1 | 1 | 0 | 0 | 0 | 1 |
| Brutus | 1 | 1 | 0 | 1 | 0 | 0 |
| Caesar | 1 | 1 | 0 | 1 | 1 | 1 |
| Calpurnia | 0 | 1 | 0 | 0 | 0 | 0 |
| Cleopatra | 1 | 0 | 0 | 0 | 0 | 0 |
| mercy | 1 | 0 | 1 | 1 | 1 | 1 |
| worser | 1 | 0 | 1 | 1 | 1 | 0 |
| ... | | | | | | |

# Which documents are returned?

# Which documents are returned?

{Anthony and Cleopatra, Hamlet}

# Boolean Query Processing (One more example; exercise)

$q$: Brutus AND Caesar AND NOT Calpurnia = 100100

$M$:

| Term Doc. | Anthony and Cleopatra | Julius Caesar | The Tempest | Hamlet | Othello | Macbeth |
|---|---|---|---|---|---|---|
| Antony | 1 | 1 | 0 | 0 | 0 | 1 |
| Brutus | 1 | 1 | 0 | 1 | 0 | 0 |
| Caesar | 1 | 1 | 0 | 1 | 1 | 1 |
| Calpurnia | 0 | 1 | 0 | 0 | 0 | 0 |
| Cleopatra | 1 | 0 | 0 | 0 | 0 | 0 |
| mercy | 1 | 0 | 1 | 1 | 1 | 1 |
| worser | 1 | 0 | 1 | 1 | 1 | 0 |
| ... | | | | | | |